



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE ENGENHARIA DE AUTOMAÇÃO E SISTEMAS
CURSO DE GRADUAÇÃO EM ENGENHARIA DE CONTROLE E AUTOMAÇÃO

Aruã Viggiano Souza

Inteligência Artificial Embarcada para Leitura de Placas de Veículos

Florianópolis
2026

Aruã Viggiano Souza

Inteligência Artificial Embarcada para Leitura de Placas de Veículos

Relatório final da disciplina DAS5511 (Projeto de Fim de Curso) como Trabalho de Conclusão do Curso de Graduação em Engenharia de Controle e Automação da Universidade Federal de Santa Catarina em Florianópolis.

Orientador: Prof. Eric Aislan Antonelo, Dr.

Supervisor: Dennis Kerr Coelho

Florianópolis

2026

Ficha de identificação da obra

A ficha de identificação é elaborada pelo próprio autor.

Orientações em:

<http://portalbu.ufsc.br/ficha>

Aruã Viggiano Souza

Inteligência Artificial Embarcada para Leitura de Placas de Veículos

Esta monografia foi julgada no contexto da disciplina DAS5511 (Projeto de Fim de Curso) e aprovada em sua forma final pelo Curso de Graduação em Engenharia de Controle e Automação

Florianópolis, <dia(número)> de <mês> de <ano(número)>.

Prof. xxxx, Dr.
Coordenador do Curso

Banca Examinadora:

[preencher somente após a defesa para a Versão Final da BU]

Prof(a). xxxx, Dr(a).
Orientador(a)
UFSC/CTC/EAS

xxxx, Eng(a).
Supervisor(a)
Empresa/Universidade xxxx

Prof(a). xxxx, Dr(a).
Avaliador(a)
Instituição xxxx

Prof. xxxx, Dr.
Presidente da Banca
UFSC/CTC/EAS

Este trabalho é dedicado aos meus colegas de classe e
aos meus queridos pais.

AGRADECIMENTOS

Inserir os agradecimentos aos colaboradores da execução do trabalho.

[illegible]

*“Texto da Epígrafe.
Citação relativa ao tema do trabalho.
É opcional. A epígrafe pode também aparecer
na abertura de cada seção ou capítulo.
Deve ser elaborada de acordo com a NBR 10520.”
(SOBRENOME do autor da epígrafe, ano)*

DECLARAÇÃO DE PUBLICIDADE

<Nome da cidade>, <dia> de <mês> de <ano>.

Na condição de representante da <instituição de realização do PFC> na qual o presente trabalho foi realizado, declaro não haver ressalvas quanto ao aspecto de sigilo ou propriedade intelectual sobre as informações contidas neste documento, que impeçam a sua publicação por parte da Universidade Federal de Santa Catarina (UFSC) para acesso pelo público em geral, incluindo a sua disponibilização *online* no Repositório Institucional da Biblioteca Universitária da UFSC. Além disso, declaro ciência de que <o/a> autor<a>, na condição de estudante da UFSC, é obrigado a depositar este documento, por se tratar de um Trabalho de Conclusão de Curso, no referido Repositório Institucional, em atendimento à Resolução Normativa nº 126/2019/CUn.

Por estar de acordo com esses termos, subscrevo-me abaixo.

<Fulano de Tal>

<Instituição de realização do PFC>

RESUMO

Instruções do padrão genérico de TCCs da BU: No Resumo são ressaltados o objetivo da pesquisa, o método utilizado, as discussões e os resultados com destaque apenas para os pontos principais. O resumo deve ser significativo, composto de uma sequência de frases concisas, afirmativas, e não de uma enumeração de tópicos. Não deve conter citações. Deve usar o verbo na voz ativa e na terceira pessoa do singular. O texto do resumo deve ser digitado, em um único bloco, sem espaço de parágrafo. O espaçamento entre linhas é simples e o tamanho da fonte é 12. Abaixo do resumo, informar as palavras-chave (palavras ou expressões significativas retiradas do texto) ou, termos retirados de thesaurus da área. Deve conter de 150 a 500 palavras. O resumo é elaborado de acordo com a NBR 6028.

Instruções da Coordenação de PFC: O Resumo deve descrever de forma sucinta: o contexto/motivação/problema tratado no PFC; a solução proposta; a implementação/desenvolvimento; a metodologia e as principais técnicas e ferramentas utilizadas; os principais resultados obtidos e a importância/impactos de tais resultados para a empresa/clientes da empresa/instituto de pesquisa. Escrever todos esses pontos de forma bem resumida e direta, e sem entrar em detalhes técnicos. O tamanho do Resumo deve ocupar praticamente esta página inteira, e num **único** parágrafo. Além disso, Resumo + Palavras-Chave não podem ultrapassar esta página.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3. *[essas palavras-chave devem obrigatoriamente ser utilizadas no Resumo]*

ABSTRACT

Resumo traduzido para outros idiomas, neste caso, inglês. Segue o formato do resumo feito na língua vernácula. As palavras-chave traduzidas, versão em língua estrangeira, são colocadas abaixo do texto precedidas pela expressão “Keywords”, separadas por ponto.

Keywords: Keyword 1. Keyword 2. Keyword 3.

LISTA DE FIGURAS

Figura 1 – Placa Radxa ROCK 3A	19
Figura 2 – Diagrama das etapas da leitura de placas	21
Figura 3 – Fases do processo CRISP-DM (Chapman et al., 2000)	28
Figura 4 – Camada convolucional (IBM Inc., 2024)	31
Figura 5 – Rede neural convolucional para classificação de imagens (Saha, 2018)	32
Figura 6 – Exemplo de detecção de objetos	33
Figura 7 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)	34
Figura 8 – Exemplo de detecção de pontos chave em um corpo humano	34
Figura 9 – Transformação de perspectiva (TheAILearner, 2023)	36
Figura 10 – Exemplo de separação de regiões por componentes conexos	38
Figura 11 – A NPU. (a) NPU de 8 <i>processing engines</i> ; (b) Fila de uma <i>processing engine</i> (Chen et al., 2020)	39

LISTA DE QUADROS

LISTA DE TABELAS

SUMÁRIO

1	INTRODUÇÃO	17
1.1	CONTEXTO DO TRABALHO	17
1.2	RECURSOS UTILIZADOS	18
1.2.1	Placa Radxa ROCK 3A	18
1.3	METODOLOGIA	19
1.3.1	Planejamento do modelo de leitura de placas	19
1.3.2	Planejamento da execução dos modelos na placa ROCK 3A	21
1.3.3	Planejamento das etapas do projeto	22
1.3.4	Preparação dos dados	22
1.3.5	Treinamento do modelo de detecção de placas	22
1.3.6	Desenvolvimento de algoritmo de pré-processamento e extração de caracteres	23
1.3.7	Geração de um <i>dataset</i> de caracteres extraídos	24
1.3.8	Treinamento de modelos de classificação de caracteres	24
1.3.9	Conversão dos modelos para formato compatível com NPU	24
1.3.10	Teste dos modelos no computador e na placa	25
2	FUNDAMENTAÇÃO TEÓRICA	26
2.1	TREINAMENTO DE MODELOS DE APRENDIZADO DE MÁQUINA	26
2.1.1	Etapas do padrão CRISP-DM	26
2.2	REDES NEURAIS E BACKPROPAGATION	28
2.3	REDES NEURAIS CONVOLUCIONAIS	30
2.4	MODELOS DE DETECÇÃO DE OBJETOS	32
2.4.1	Modelos YOLO	33
2.4.2	Detecção de pontos chave	34
2.5	TÉCNICAS DE TRANSFORMAÇÃO DE IMAGEM PARA VISÃO COMPUTACIONAL	35
2.5.1	Transformação de perspectiva	35
2.5.2	Limiarização adaptativa	36
2.5.3	Segmentação de caracteres	37
2.5.4	Filtragem de componentes conexos	37
2.6	NEURAL PROCESSING UNIT	38
2.7	QUANTIZAÇÃO DE REDES NEURAIS	39
3	DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS	41
3.1	DESCRIÇÃO DA EMPRESA	41
3.2	MOTIVAÇÃO	42
3.3	ANÁLISE DO PROBLEMA	43
3.3.1	Criação do algoritmo de leitura automática de placas de carros	43

3.3.2	Teste do algoritmo desenvolvido na placa Rock 3A	44
3.4	REQUISITOS DA SOLUÇÃO	44
4	SOLUÇÃO PROPOSTA E METODOLOGIA UTILIZADA	45
4.1	ANÁLISE DOS DADOS	45
4.1.1	Características do objeto	45
4.1.2	Características do <i>dataset</i> utilizado	45
4.2	<i>PIPELINE</i> DE LEITURA DE PLACAS	45
4.3	MODELO DE DETECÇÃO DE OBJETOS	45
4.4	ALGORITMO DE EXTRAÇÃO DE CARACTERES	45
4.5	MODELOS DE CLASSIFICAÇÃO DE CARACTERES	45
4.6	METODOLOGIA DE CONSTRUÇÃO	45
5	DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS .	46
5.1	PREPARAÇÃO DOS DADOS	46
5.2	TREINAMENTO DO MODELO DE DETECÇÃO DE PLACAS	46
5.3	IMPLEMENTAÇÃO DO ALGORITMO DE EXTRAÇÃO DE CARACTERES	46
5.4	CRIAÇÃO DO DATASET DE CARACTERES	46
5.5	TREINAMENTO DOS MODELOS DE CLASSIFICAÇÃO DE CARACTERES	46
5.6	QUANTIZAÇÃO E TRANSFORMAÇÃO DOS MODELOS PARA FORMATO COMPATÍVEL COM O <i>HARDWARE</i>	46
5.7	PREPARAÇÃO DO <i>HARDWARE</i>	46
5.8	TESTES REALIZADOS	46
6	CONCLUSÃO	47
	REFERÊNCIAS	48
	APÊNDICE A – DESCRIÇÃO 1	50
	ANEXO A – DESCRIÇÃO 2	51

1 INTRODUÇÃO

Nos últimos anos a leitura automática de placas de veículos se tornou uma aplicação presente no cotidiano de muitos. Inúmeros estacionamentos de shoppings, supermercados e outros estabelecimentos de grande ou médio porte descobriram nessa tecnologia uma forma de melhorar a experiência do cliente que utiliza o estacionamento. O conceito é simples: quando o carro se aproxima da entrada a placa é imediatamente detectada e associada ao *ticket*. Então, quando o cliente se aproxima da cancela de saída, o sistema lê a placa novamente e já consegue verificar qual *ticket* está associado àquele placa e se ele já foi pago ou está na tolerância. Se o *ticket* está liberado a cancela abre e deixa o carro passar sem que o motorista precise aproximar o *ticket* do leitor.

Esse sistema gera as seguintes vantagens para os motoristas e para o próprio estabelecimento:

- Agiliza a saída dos carros, evitando formação de filas
- Permite que *tickets* perdidos possam ser recuperados digitalmente
- Cria a possibilidade de o estabelecimento cadastrar certas placas para receber acesso automático, sem precisar de *ticket*, facilitando o acesso dos funcionários.

Esses benefícios facilitam a vida dos usuários do estacionamento, melhorando a experiência daqueles que frequentam o estabelecimento e consequentemente agregando valor ao negócio.

Essa é apenas uma das possíveis aplicações da leitura automática de placas de veículo, que pode ser utilizada também em aplicações de segurança, auditoria, controle de tráfego e controle de acesso.

Nesse contexto, o desenvolvimento de modelos eficazes, rápidos e leves para leitura automática de placas é de extremo interesse econômico, pois pode permitir a aplicação desses modelos em dispositivos de baixo custo e aumentar sua adoção em diversas aplicações. Isso é o que esse trabalho se propõe a realizar.

1.1 CONTEXTO DO TRABALHO

O interesse nesse trabalho surgiu no contexto de um condomínio autônomo focado em aluguel para estadias de curto prazo. Nesse condomínio os hóspedes reservam os apartamentos em sites como *Booking* ou *AirBnB*. Os administradores do apartamento então cadastram a reserva em um sistema que libera automaticamente a entrada do hóspede no condomínio durante o período da reserva.

O acesso do hóspede ao condomínio se dá por reconhecimento facial, mas no estacionamento é utilizada uma câmera com leitura de placas de veículo integrada.

Só de aproximar seu carro, o portão já se abre, pois a placa é reconhecida como pertencente à um hóspede com reserva ativa e quando a estadia chega ao fim, o acesso é imediatamente revogado.

A câmera que torna isso possível é um dispositivo da Intelbras, modelo VIP 5460 LPR IA, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500, longe de um preço acessível para aplicações de menor escala ou casos em que a quantidade de dispositivos necessários é maior.

Esse trabalho, então, se propõe a realizar uma prova de conceito: Criar um modelo de leitura automática de placas e executá-lo em uma placa de baixo custo. Esse seria o primeiro passo para o desenvolvimento de um protótipo barato de câmera com leitura automática de placas.

1.2 RECURSOS UTILIZADOS

Os recursos disponíveis para a realização desse projeto foram:

- Computador com CPU Intel Core i7 e GPU NVIDIA GeForce RTX 3060
- Placa Radxa ROCK 3A com Rockchip RK3568
- Micro SD 32 GB

1.2.1 Placa Radxa ROCK 3A

A placa ROCK 3A da Radxa foi o hardware disponibilizado para executar o modelo de leitura automática de placas a ser desenvolvido. Essa placa é equipada com um chip Rockchip RK3568, o qual possui os seguintes componentes:

- CPU Quad-core Arm® Cortex™-A55, 64-bit, 2GHz
- GPU Arm® Mali™ G52
- NPU 0.8 TOPs, INT8

A figura 1 mostra a vista superior da placa ROCK 3A.

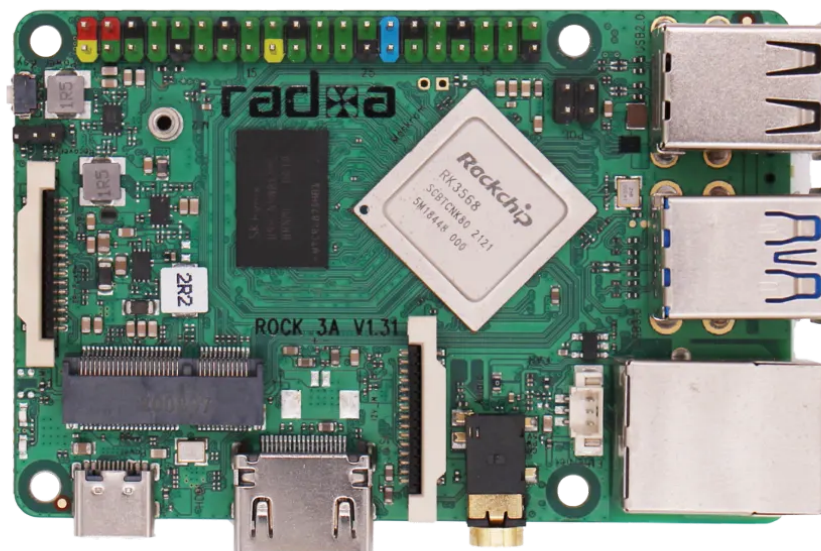


Figura 1 – Placa Radxa ROCK 3A

1.3 METODOLOGIA

A metodologia para o desenvolvimento do projeto começou com o planejamento da solução. Para isso, foi necessário quebrar o problema em problemas menores que podem ser resolvidos mais facilmente.

A primeira quebra a ser feita é a separação do problema de leitura de placas de carro do problema do hardware que executará a solução. Primeiro é necessário atingir um resultado satisfatório em ambiente de desenvolvimento para só então evoluir para o ambiente final da prova de conceito.

1.3.1 Planejamento do modelo de leitura de placas

O *input* com o qual a solução irá lidar é uma imagem na qual aparece a placa de um carro. Porém, apenas a placa em si nos interessa, sendo todo o resto da imagem informação desnecessária para o nosso objetivo. Portanto, o primeiro problema consiste em transformar a imagem que contém a placa de um carro em uma imagem que contém *somente* a placa de um carro. Todo o resto deve ser removido.

A partir da imagem da placa do carro, o próximo desafio é identificar onde estão as letras e os números. Modelos de classificação de imagens, principalmente de

imagens de caracteres, são extremamente eficazes, mas precisam que a imagem contenha única e exclusivamente o caractere a ser classificado. Dessa forma, a imagem contendo a placa inteira precisa ser quebrada em pedaços, cada um contendo um caractere.

Para que essa divisão da placa em caracteres tenha sucesso, é necessário saber como os caracteres estão dispostos pela placa, o que varia conforme o tipo da placa. No Brasil existem dois de *layout* de placa permitidos: O tipo antigo, ainda em circulação, e o tipo Mercosul, que passou a ser obrigatório para veículos novos no Brasil em 2020.

Por fim, o último problema é classificar cada imagem de caractere. Tendo as imagens classificadas, e sendo a sua ordem na placa conhecida, é possível inferir a placa do veículo. Com isso, o problema do modelo estaria resolvido.

A conclusão é que o problema do modelo de leitura de placas de carro pode ser dividido nos seguintes três problemas:

- Detecção e classificação da placa
- Extração dos caracteres
- Classificação dos caracteres

A detecção e classificação são uma única etapa pois modelos de detecção de objetos costumam ter a capacidade de detectar objetos de múltiplas classes e identificar qual é a classe de cada objeto detectado. Isso permite que a detecção da placa e a classificação do tipo de placa sejam feitos de uma só vez.

Para que a extração e classificação dos caracteres funcione bem, é vital que haja um pré-processamento da imagem da placa extraída pela detecção, com objetivo de remover ao máximo o ruído da imagem e deixando-a o mais limpa possível.

O diagrama da figura 2 mostra todas essas etapas assim como suas entradas e saídas.

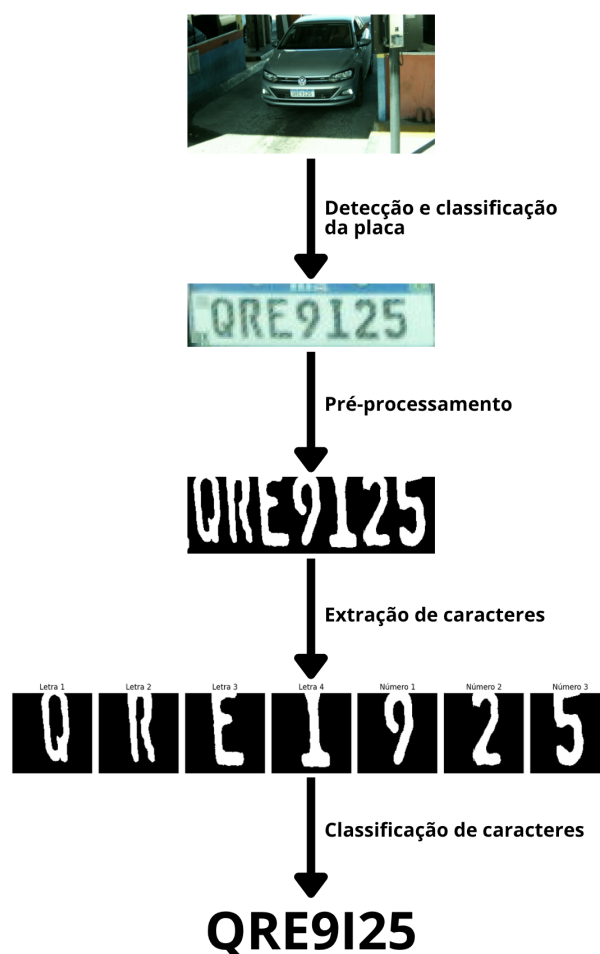


Figura 2 – Diagrama das etapas da leitura de placas

1.3.2 Planejamento da execução dos modelos na placa ROCK 3A

A placa ROCK 3A é equipada com uma NPU (*Neural Processing Unit*), que em tese executa modelos de redes neurais de forma mais eficiente que uma CPU. Portanto, nosso objetivo é que os modelos sejam executados na NPU.

Para isso são necessárias duas etapas principais: Preparação da placa e preparação dos modelos.

A preparação da placa envolve:

- Instalar o sistema operacional
- Instalar todo o software necessário para inferência
- Ativar a NPU

A preparação dos modelos consiste basicamente em convertê-los para o formato RKNN, que é o formato aceito para a NPU do ROCK 3A.

1.3.3 Planejamento das etapas do projeto

Tendo o planejamento do modelo e da execução na placa, podemos agora pensar nas etapas do projeto. Essas etapas devem abarcar o desenvolvimento do modelo, a execução no *hardware* do projeto e o teste final dos resultados.

As etapas sequenciais, portanto são:

1. Preparação dos dados a serem utilizados para treinamento e teste do modelo
2. Treinamento de um modelo de detecção de placas (apenas identifica o tipo e a posição da placa)
3. Desenvolvimento de um algoritmo de pré-processamento e extração de caracteres
4. Geração de um dataset de caracteres extraídos
5. Treinamento de modelos de classificação de caracteres
6. Conversão dos modelos para formato compatível com NPU
7. Teste dos modelos no computador e na placa

1.3.4 Preparação dos dados

A preparação inicial dos dados envolveu principalmente dois procedimentos: Separação e organização.

A separação consistiu em separar os dados em três subgrupos, sendo eles **treino**, **validação** e **teste**. Os subgrupos possuem a função de impedir que dados utilizados no desenvolvimento da solução sejam também utilizados para avaliar a qualidade da mesma, de forma a não gerar um viés nos testes.

A organização, por outro lado, foi feita tendo em mente o algoritmo de treinamento do modelo de detecção de placas, que requer que os dados estejam organizados em uma dada estrutura, como veremos mais à frente. Além disso, o *dataset* de teste foi organizado para os testes da solução completa, a serem realizados no final do projeto.

1.3.5 Treinamento do modelo de detecção de placas

O modelo de detecção de placas possui três funções:

- Detectar os limites da placa na imagem (superior, inferior, esquerdo e direito)
- Detectar as coordenadas dos quatro cantos da placa
- Detectar a classe da placa entre duas possibilidades: Brasil e Mercosul

Sendo que a classe "Brasil" refere-se ao modelo antigo de placas brasileiras, enquanto "Mercosul" refere-se ao modelo novo, adotado em todo o bloco Mercosul.

Para essa funcionalidade o método utilizado foi treinar um modelo pré-treinado de detecção de objetos, adaptando-o para a necessidade específica do projeto. A biblioteca Ultralytics, do Python, oferece uma interface para esse tipo de treinamento de modelos de detecção de objetos. Essa interface requer uma organização específica dos dados, que foi realizada na etapa anterior.

1.3.6 Desenvolvimento de algoritmo de pré-processamento e extração de caracteres

O modelo de detecção de placas de carro entrega a posição da placa na imagem e as coordenadas dos quatro cantos. A partir dessa informação é trivial cortar apenas a parte da imagem que contém a placa. Isso já ajuda bastante no processo de extração dos caracteres, mas não é suficiente. Uma questão importante é que em muitos casos a placa possui certa rotação ou perspectiva, o que pode atrapalhar bastante. Para corrigir isso é possível aplicar uma transformação linear com base nas coordenadas dos quatro cantos que corrige essa distorção.

Além disso, as placas possuem partes que não nos interessam. Apenas os caracteres são relevantes para a leitura, sendo todo o resto (parte superior, inferior, laterais) descartável.

Um outro fato importante é que é possível saber previamente quais caracteres são letras e quais são números com base em sua posição na placa. Tanto a placa Brasil quanto a Mercosul possui uma sequência bem definida de letras e números, de forma que basta saber qual é a classe da placa para que se saiba onde as letras e os números estão posicionados.

Com base nesses fatos, o algoritmo de pré-processamento e extração de caracteres deve:

- Corrigir a perspectiva e rotação da placa para que ela fique reta e com dimensões padronizadas
- Cortar os cantos que não possuem informações relevantes
- Fazer uma limpeza e remover todo o ruído desnecessário
- Dividir a placa em pedaços, cada um contendo um caractere
- Padronizar as dimensões das imagens de caracteres
- Separar os caracteres extraídos em letras e números com base na posição

1.3.7 Geração de um *dataset* de caracteres extraídos

Para que seja possível treinar os modelos de classificação dos caracteres é necessário criar um *dataset* de caracteres. Tendo sido realizadas as etapas anteriores, basta aplicar a detecção de placa e extração de caracteres em cada imagem dos dados de treinamento para criar um *dataset* de caracteres. Como sabemos previamente quais placas são do tipo Brasil e quais são Mercosul, assim como também sabemos quais caracteres são números e quais são letras, é possível criar *datasets* separados para cada categoria, resultando em quatro *datasets*:

- Números Brasil
- Letras Brasil
- Números Mercosul
- Letras Mercosul

Sendo que cada um destes ainda é separado em treino e validação.

1.3.8 Treinamento de modelos de classificação de caracteres

Com o *dataset* de caracteres pronto, basta criar uma arquitetura de modelo de visão computacional e treiná-lo com esses dados. Como os dados já vêm pré-processados e a tarefa de classificação de caracteres não é particularmente complexa, basta uma arquitetura simples de redes neurais convolucionais para resolver esse problema.

1.3.9 Conversão dos modelos para formato compatível com NPU

O formato compatível com a NPU do ROCK 3A é o formato RKNN. A própria documentação da placa indica uma biblioteca chamada *rknn toolkit* que oferece a *API* tanto para conversão dos modelos para esse formato quanto para inferência com esses modelos. Porém, a conversão não é tão trivial, pois precisa ser feita em um ambiente *linux* de uma determinada distribuição e versão. Além disso, os modelos precisam ser exportados para o formato ONNX antes de serem convertidos, pois o *rknn toolkit* faz a conversão entre esses dois formatos.

Além desses requisitos obrigatórios para a conversão, existe um requisito opcional, porém recomendado. Este é que durante a conversão para RKNN os modelos sejam quantizados para *uint8*, ou seja, que seus pesos sejam convertidos de números reais do tipo *float* de 32 *bits* para números inteiros positivos de 8 *bits*. Isso é recomendado pois intensifica a performance da NPU.

Para a quantização é necessário criar um *dataset* de quantização. Esse *dataset* só precisa conter dados de entrada, não sendo necessários os rótulos.

1.3.10 Teste dos modelos no computador e na placa

Uma vez que os modelos estão prontos e convertidos é possível testá-los tanto no computador quanto na placa. No computador ainda é possível testá-los na CPU e na GPU. Para avaliar a performance nos diferentes cenários, vamos usar o *dataset* de teste que foi gerado no começo e avaliar em quantas imagens o modelo acerta a placa do carro.

Porém, a principal diferença entre os locais de execução não deve vir no percentual de acerto, e sim no tempo de inferência. Portanto, é necessário medir o tempo, tanto do modelo de detecção de placa quanto nos modelos de classificação de caracteres.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo é discutida a fundamentação teórica dos principais conceitos, técnicas e tecnologias utilizados no projeto, incluindo:

- Treinamento de modelos de aprendizado de máquina
- Redes neurais e *backpropagation*
- Redes neurais convolucionais
- Modelos de detecção de objetos
- Técnicas de transformação de imagens para visão computacional
- *Neural Processing Unit*
- Quantização de redes neurais

Essa fundamentação forma a base necessária para a criação, desenvolvimento e validação do algoritmo de leitura de placas de carro desenvolvido nesse projeto, além de sua execução no *hardware* específico disponível.

2.1 TREINAMENTO DE MODELOS DE APRENDIZADO DE MÁQUINA

O processo de treinamento de modelos de aprendizado de máquina nada mais é do que uma variação específica do processo padrão de mineração de dados, que consiste no ato de utilizar algoritmos para extrair informações relevantes de um banco de dados (Fayyad; Piatetsky-Shapiro; Smyth, 1996).

Este processo já possui uma forma bem definida e aceita conhecida como CRISP-DM (*Cross-Industry Standard Process for Data Mining*) (Chapman et al., 2000).

O treinamento de modelos de aprendizado de máquina se enquadra como um processo de mineração de dados, pois nada mais é do que encontrar um modelo que, de alguma forma, represente o relacionamento entre diferentes aspectos do conjunto de dados. O modelo, nesse caso, representa a informação que se deseja extrair dos dados.

Um bom modelo descreverá com precisão o relacionamento entre os aspectos desejados do conjunto de dados e será portanto útil para extrair novas informações de novos dados.

2.1.1 Etapas do padrão CRISP-DM

O padrão CRISP-DM nada mais é que uma sequência de etapas a partir das quais é possível extrair informações úteis de um banco de dados, sendo que cada etapa é definida por tarefas e um resultado esperado.

As etapas são:

1. **Entendimento do problema**

A primeira fase consiste em entender as características do problema e os requisitos da solução, criando assim uma definição de problema de mineração de dados e um plano preliminar para atingir os objetivos (Chapman et al., 2000).

2. **Entendimento dos dados**

A fase seguinte envolve se familiarizar com os dados à disposição. Isso envolve descobrir *insights* iniciais nos dados, detectar problemas de qualidade dos dados e identificar subsets interessantes (Chapman et al., 2000).

3. **Preparação dos dados**

Essa etapa consiste nas atividades necessárias para construir o *dataset* final que será utilizado a partir dos dados brutos. Pode envolver diversas tarefas como limpeza dos dados, seleção de atributos relevantes e transformação dos dados (Chapman et al., 2000).

4. **Modelagem**

A modelagem envolve selecionar, aplicar e calibrar técnicas diversas de modelagem, buscando encontrar um modelo ótimo para o problema que se deseja resolver (Chapman et al., 2000).

5. **Avaliação**

Após chegar em um ou mais modelos que aparentam ser apropriados, uma avaliação final deles deve ser feita para concluir se atendem aos requisitos de negócio. Ao final dessa avaliação deve-se chegar em uma conclusão relativa ao uso ou não de cada modelo para o objetivo final (Chapman et al., 2000).

6. **Deployment**

Por fim, a última etapa é o momento em que o conhecimento obtido é preparado para uso pelo cliente do projeto. Essa preparação depende muito dos requisitos, podendo ser em forma de um relatório ou de um sistema automatizado de mineração de dados (Chapman et al., 2000).

A figura 3 mostra um diagrama do processo e demonstra como algumas das etapas possuem uma relação de troca, de forma que o conhecimento ou os desafios encontrados em uma dada etapa podem fazer com que seja necessário voltar para uma etapa anterior.

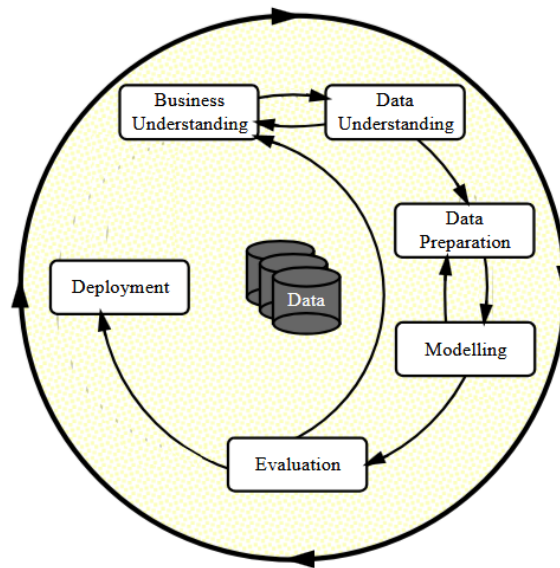


Figura 3 – Fases do processo CRISP-DM (Chapman et al., 2000)

2.2 REDES NEURAIS E BACKPROPAGATION

As redes neurais artificiais e o algoritmo de *backpropagation* são alguns dos fundamentos mais importantes da inteligência artificial moderna. As redes neurais permitem a criação de modelos capazes de aproximar relações não lineares e complexas entre dados, sendo que o limite da complexidade dessas relações se dá apenas pela limitação computacional ou do volume de dados utilizados no ajuste do modelo. Já o algoritmo de *backpropagation* permite ajustar o modelo de redes neurais aos dados, minimizando uma função de erro.

Esses conceitos são utilizados para criar modelos de inteligência artificial voltados para visão computacional, processamento de linguagem natural, séries temporais, IA generativa e inúmeras outras aplicações.

O tipo clássico de redes neurais é composto por uma série de camadas de processamento. Cada camada processa a saída da camada anterior em sequência até chegar no *output* final. A saída $a_{(c)}$ de uma camada c é definida por:

$$a_{(c)} = f(z_{(c)}) \quad (1)$$

Sendo que f é a função de ativação da camada e $z_{(c)}$ é definido como:

$$z_{(c)} = W_{(c)} \times a_{(c-1)} + b_{(c)} \quad (2)$$

Sendo:

- $W_{(c)}$ a matriz de pesos da camada
- $a_{(c-1)}$ a saída da camada anterior

- $b_{(c)}$ o vetor de viés da camada

A função de ativação f é responsável por conferir não-linearidade à camada, pois sem ela a camada não passaria de uma transformação linear capaz apenas de representar relações lineares entre os dados, limitando muito sua aplicabilidade.

Existem diversas funções de ativação que podem ser usadas, mas as mais comuns são:

- **Sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tangente hiperbólica:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ReLU (*Rectified Linear Unit*):** $\text{ReLU}(x) = \begin{cases} x ; & x > 0 \\ 0 ; & x \leq 0 \end{cases}$

A matriz de pesos $W_{(c)}$ e o vetor de viés $b_{(c)}$ são compostos por valores ajustáveis que devem ser modificados até que o modelo esteja satisfatório.

Para que os pesos do modelo sejam ajustados é utilizado o algoritmo de *back-propagation* para calcular o gradiente da função de erro em relação aos pesos da rede. A função de erro varia conforme o objetivo daquela rede neural e da função de ativação da última camada.

Para calcular o gradiente, então, é necessário calcular o delta $\delta_{(c)}$ de cada camada.

Para a camada de saída da rede (C) o delta é dado por:

$$\delta_{(C)} = \hat{y} - y \quad (3)$$

Sendo que $\hat{y}$ é o valor predito e y é o valor esperado (verdadeiro).

Para as demais camadas o delta é definido como:

$$\delta_{(c)} = W_{(c+1)}^T \times \delta_{c+1} \odot f'(z_{(c)}) \quad (4)$$

E por fim, o gradiente da função de erro L em relação aos pesos fica:

$$\frac{\partial L}{\partial W_{(c)}} = \delta_{(c)} \times a_{(c-1)}^T \quad (5)$$

$$\frac{\partial L}{\partial b_{(c)}} = \delta_{(c)} \quad (6)$$

Nota-se que, enquanto a saída de uma camada é calculada com base na saída da camada anterior, o gradiente de uma camada é calculado com base no delta da camada seguinte. Essa propagação retroativa dos erros é o motivo de o algoritmo de cálculo do gradiente ser chamado de *backpropagation* (Rumelhart; Hinton, Geoffrey E.; Williams, 1986).

O último fator importante a ser discutido no que diz respeito às redes neurais artificiais é o algoritmo de atualização dos pesos com base no gradiente. Existem diversos algoritmos diferentes que podem ser utilizados, porém o mais clássico e simples é o SGD (*stochastic Gradient Descent*), no qual a um peso $v_{(c)}$ é atribuído:

$$v_{(c)} \leftarrow v_{(c)} - \eta \frac{\partial L}{\partial v_{(c)}} \quad (7)$$

Onde η é a taxa de aprendizagem.

2.3 REDES NEURAI CONVOLUCIONAIS

A eficácia de redes neurais convolucionais (CNNs) para tarefas de classificação e segmentação de imagens já é comprovada (Krizhevsky; Sutskever; Hinton, Geoffrey E, 2012), sendo uma solução amplamente utilizada em diversas aplicações.

Essa modalidade de rede neural possui como característica as camadas convolucionais, que possui vantagens em relação as camadas *fully connected* tradicionais para tarefas de interpretação de imagens ou mesmo de sinais (LeCun et al., 1998).

A limitação das camadas *fully connected* para esse tipo de tarefa se dá pelo fato de que esse tipo de camada produz sua saída com base no relacionamento entre todos os *inputs* da camada uns com os outros. Isso é ótimo para achar relações entre diferentes atributos quando se trabalha com dados tabulares, por exemplo, mas não é tão bom para encontrar características específicas que podem estar localizadas em diferentes partes de uma imagem.

Para esse tipo de tarefa as camadas convolucionais são mais apropriadas, pois utilizam conectividade local e compartilhamento de pesos, reduzindo drasticamente a quantidade de parâmetros necessários para se ter um resultado satisfatório.

A camada convolucional funciona deslizando pelo *input* um filtro chamado *kernel*, que nada mais é do que uma matriz de pesos de dimensão pequena, como 3x3 ou 5x5. Em cada posição do *kernel* é então feita uma transformação linear de um conjunto local de valores utilizando os pesos do *kernel*.

Fazendo isso em todo o *input* obtêm-se um mapa de características, que destaca padrões identificados em cada parte do *input*. Esse mapa de características pode então ser alimentado para outra camada convolucional e assim por diante. Além disso, para cada *input* pode-se (e deve-se) ter múltiplos *kernels*, de forma a gerar múltiplos mapas de características em cada camada, cada um destacando aspectos distintos do *input*. A imagem 4 exemplifica o funcionamento da camada convolucional.

A equação da convolução 2D discreta utilizada nas camadas convolucionais para imagens é:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j) \quad (8)$$

Sendo que I é a imagem e K é o *kernel*.

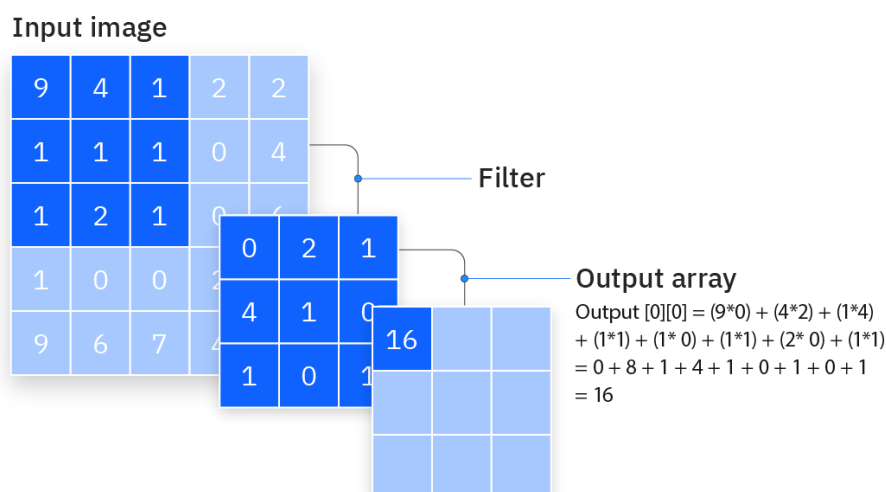


Figura 4 – Camada convolucional (IBM Inc., 2024)

Redes neurais convolucionais também costumam utilizar camadas de *pooling* após cada camada convolucional. Essa camada utiliza um método de subamostragem para reduzir a dimensão de seu *input*. Suas duas formas mais comuns são o *max pooling* e o *average pooling*.

A ideia do *pooling* é dividir o *input* em janelas não sobrepostas e transformar cada janela em um único valor escalar, aplicando sobre a janela o cálculo da média ou extraindo seu valor máximo, a depender da técnica utilizada.

As camadas convolucionais possuem a tarefa de extrair atributos da imagem, e são ótimos em identificar atributos específicos independente da região da imagem em que estejam localizados. Após a extração dos atributos, porém, ainda é necessário utilizá-los de alguma forma. Em tarefas de classificação de imagens é muito comum alimentar os atributos extraídos para camadas *fully connected*, que aprendem padrões dos atributos e dos relacionamentos entre eles e utilizam isso para classificar a imagem.

A forma padrão de um modelo de redes neurais convolucionais para classificação de imagens, portanto, é uma sequência de camadas convolucionais e camadas de *pooling*, seguidas por uma sequência de camadas *fully connected* que culminam no *output* do modelo. A figura 5 mostra um diagrama que exemplifica a arquitetura de uma rede neural convolucional para classificação de imagens.

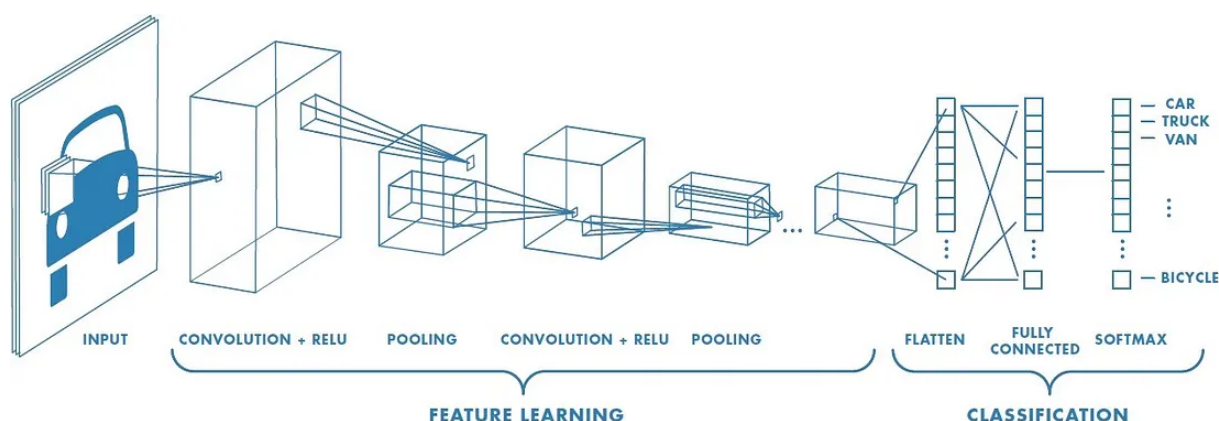


Figura 5 – Rede neural convolucional para classificação de imagens (Saha, 2018)

2.4 MODELOS DE DETECÇÃO DE OBJETOS

A tarefa de detecção de objetos é um campo desafiador dentro da área mais ampla de visão computacional. Essa tarefa consiste em identificar objetos específicos dentro de uma imagem, incluindo sua classe e localização dentro da imagem. É, portanto, uma tarefa mais complexa que a classificação de imagens, que se preocupa simplesmente em atribuir uma classe à imagem toda, enquanto a detecção de objetos deve extrair da imagem regiões retangulares chamadas *bounding boxes* e atribuir uma classe a cada uma dessas regiões.

Isso permite diversas aplicações como:

- Rastrear a movimentação de um objeto ao longo de um vídeo
- Verificar quantas instâncias de um dado objeto existem na imagem
- Extrair da imagem original uma nova imagem contendo apenas o objeto de interesse

A figura 6 exemplifica a tarefa de detecção de objetos em visão computacional.

Os modelos mais comuns de detecção de objetos costumam ser divididos em duas categorias: detectores de duas etapas e detectores de uma etapa (Zou et al., 2019). Os modelos de duas etapas primeiro dividem a imagem em regiões candidatas, onde a probabilidade de haver objetos é alta, e posteriormente gera as *bounding boxes* e classifica as regiões. Modelos de duas etapas costumam ser mais precisos, mas o tempo de inferência é mais alto.

Modelos de uma etapa, por outro lado, olham para a imagem inteira e tentam classificar as regiões e gerar as *bounding boxes* diretamente. Esses modelos possuem a vantagem de serem mais rápidos, portanto mais apropriados para aplicações de tempo real.

Como tarefas de detecção de objetos são muito utilizadas em aplicações de tempo real, a velocidade de inferência é uma métrica muito importante na avaliação

desse tipo de modelo, além da precisão do resultado. Nessa linha, modelos que colocam grande ênfase na velocidade de inferência ganharam grande tração no campo da detecção de objetos. Um desses modelos (ou família de modelos) é o YOLO, do qual falaremos mais para a frente.

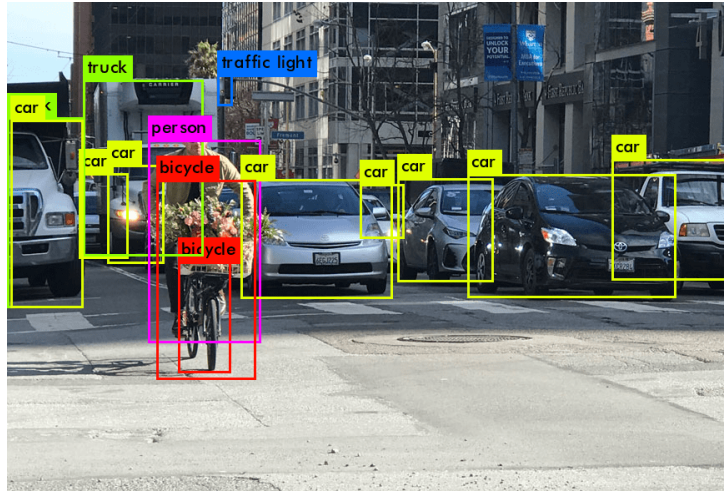


Figura 6 – Exemplo de detecção de objetos

2.4.1 Modelos YOLO

Os modelos da família YOLO (*You Only Look Once*) são modelos muito rápidos de detecção em uma etapa, portanto apropriados para aplicações de tempo real. Quando os modelos YOLO surgiram, apenas modelos de duas etapas eram utilizados para detecção de objetos com redes neurais, mas a inovação proposta com os modelos de uma etapa se mostrou extremamente eficiente e se tornou amplamente utilizada (Redmon et al., 2016).

Modelos YOLO tratam a tarefa de detecção de objetos como uma única tarefa de regressão, diferente dos modelos de duas etapas que se baseiam em uma *pipeline* mais complexa. Ele funciona dividindo a imagem em um grid $S \times S$, e cada célula do grid é responsável por detectar os objetos cujo centro está dentro da célula. Cada célula então prevê B *bounding boxes*, sendo que cada *bounding box* é representada por 5 valores: As coordenadas (x, y) do centro relativo à célula, a altura, a largura e a confiança. A confiança representa a *IoU* (*Intersection over Union*) da *bounding box* prevista com a verdadeira caso haja um objeto na célula. Se não houver objeto a confiança deve ser igual a zero (Redmon et al., 2016).

$$IoU = \frac{\text{Area de Interceccao}}{\text{Area de Uniao}} \quad (9)$$

Cada célula também produz C probabilidades de classe, cada uma representando a probabilidade de aquela célula conter um objeto de uma dada classe.

A figura 7 mostra uma representação do funcionamento da rede YOLO.

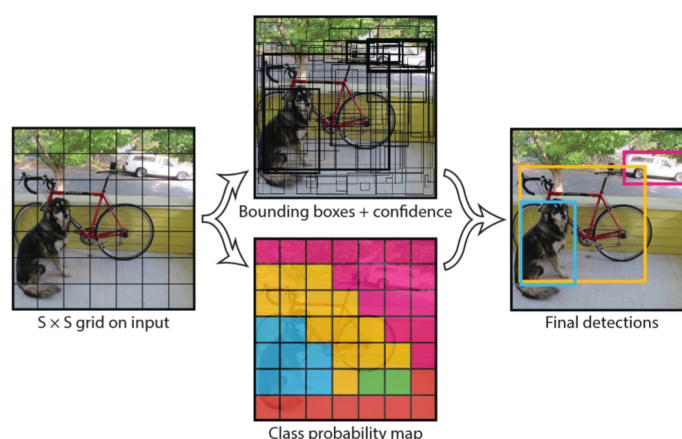


Figura 7 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)

2.4.2 Detecção de pontos chave

Dentro da área da detecção de objetos à também uma subdivisão chamada detecção de pontos chave. Essa tarefa consiste em, além de detectar um objeto, detectar também a posição de uma série de pontos importantes do objeto. Esses pontos podem ser importantes em vários contextos pois oferecem informação adicional a respeito da posição ou formato do objeto.

Uma aplicação muito comum da detecção de pontos chave é estimar a posição de seres humanos. Ao identificar a posição de diversas partes do corpo como ombros, cotovelos, mãos, etc. relativas umas às outras, é possível inferir a posição da pessoa e se ela está tomando certas ações, como andar ou sentar.

A figura 8 mostra um exemplo de detecção de uma série de pontos chave no corpo humano.

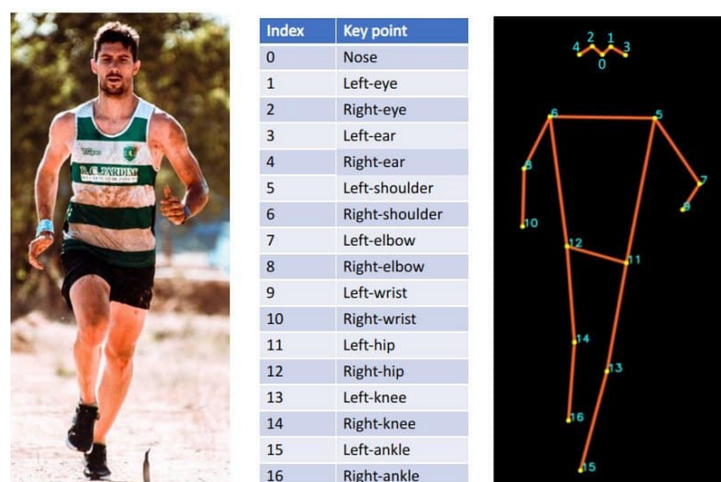


Figura 8 – Exemplo de detecção de pontos chave em um corpo humano

2.5 TÉCNICAS DE TRANSFORMAÇÃO DE IMAGEM PARA VISÃO COMPUTACIONAL

Inúmeros algoritmos de manipulação de imagens são utilizados em visão computacional desde muito antes do surgimento das redes neurais. Alguns desses algoritmos ainda são extremamente relevantes, especialmente no tratamento de imagens que possuem padrões conhecidos.

Essas técnicas são úteis em especial para extrair características das imagens e prepará-las para serem alimentadas por modelos de inteligência artificial, removendo ruídos e destacando as características mais relevantes da imagem.

As principais técnicas utilizadas nesse projeto foram:

- Transformação de perspectiva
- Limiarização adaptativa
- Segmentação de caracteres
- Filtragem de componentes conexos

2.5.1 Transformação de perspectiva

A transformação de perspectiva é uma operação utilizada para corrigir a perspectiva de uma figura 2D dentro de uma imagem. Ela permite que objetos inclinados ou rotacionados sejam alinhados com base nas coordenadas dos quatro cantos do objeto. Esse tipo de transformação não preserva paralelismo, comprimento e ângulo, mas preserva colinearidade e incidência (TheAILearner, 2023).

A transformação de perspectiva pode ser expressada como:

$$\begin{bmatrix} t_j x' \\ t_j y' \\ t_j \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (10)$$

Sendo que x' e y' são os pontos transformados, ou seja, as coordenadas (x, y) dos pontos na imagem de saída, enquanto x e y são os pontos originais. O termo h_i é um fator de escala.

A matriz de transformação

$$M = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \quad (11)$$

é formada por três componentes: A matriz $\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}$, responsável por definir transformações como rotação e escala, o vetor $\begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$, responsável por definir a translação e o vetor de projeção $\begin{bmatrix} c_1 & c_2 \end{bmatrix}$.

Como a matriz de transformação possui 8 incógnitas, são necessárias 8 equações para que seja resolvida. Para isso são selecionados quatro pontos (x, y) na imagem original e quatro pontos (x', y') na imagem de saída. Dessa forma é possível obter a matriz de transformação capaz de mapear todos os pontos da imagem original para a imagem final.

A figura 9 exemplifica o que ocorre com uma imagem na transformação de perspectiva.

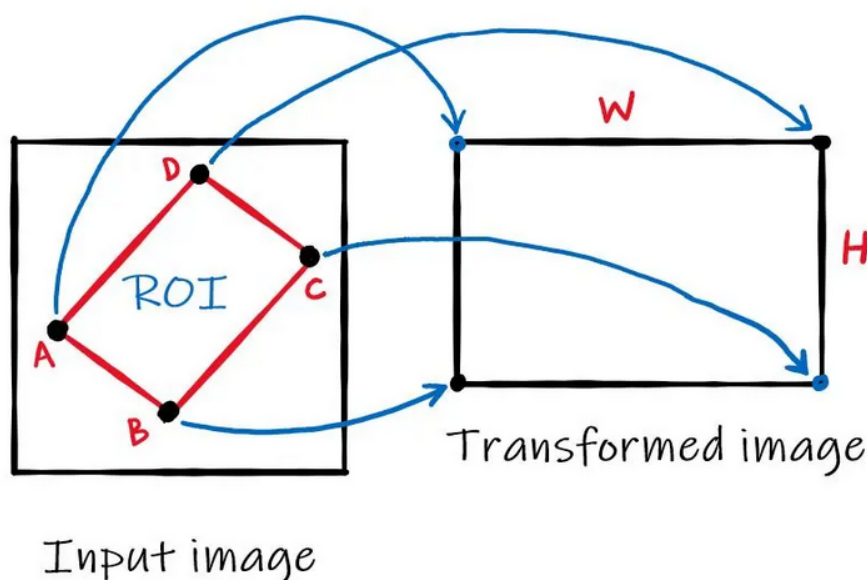


Figura 9 – Transformação de perspectiva (TheAILearner, 2023)

2.5.2 Limiarização adaptativa

A limiarização é uma técnica muito utilizada em visão computacional para criar máscaras a partir de imagens. Ela consiste em definir um limiar e classificar todos os pixels da imagem em 0 (preto) ou um (branco), dependendo se o valor original do pixel ficava acima ou abaixo do limiar definido. Uma aplicação muito comum desse conceito em visão computacional é a separação de um objeto do fundo da imagem.

Independente da aplicação, a escolha do valor do limiar impacta profundamente a imagem binarizada resultante. Uma escolha mal feita pode fazer com que partes significativas do objeto sejam apagadas ou com que ruídos do fundo se misturem ao objeto. Portanto, a aplicação de uma boa técnica para selecionar o limiar se faz necessária.

Um outro fator relevante a se considerar é que em muitos casos a iluminação faz com que certas partes da imagem sejam mais escuras que outras. Isso faz com que seja interessante utilizar um limiar diferente em diferentes partes da imagem, de forma a evitar distorções por diferença na iluminação.

Um método para determinação desse limiar variável é o método de Sauvola (Sauvola; Pietikäinen, 2000), que determina um limiar ideal para cada pixel da imagem usando os pixels vizinhos como base.

O método funciona calculando a média e desvio padrão local para cada pixel com base em uma janela. A partir disso é então calculado o limiar ideal para aquele pixel.

A média local para um dado pixel de coordenadas (x, y) na imagem I é definida como:

$$\mu(x, y) = \frac{1}{W^2} \sum_{i=-\frac{W}{2}}^{\frac{W}{2}} \sum_{j=-\frac{W}{2}}^{\frac{W}{2}} I(x + i, y + j) \quad (12)$$

Sendo que W é o tamanho da janela escolhido.

Já o desvio padrão local desse mesmo pixel é definido como:

$$\sigma(x, y) = \sqrt{E[I^2](x, y) - \mu(x, y)^2} \quad (13)$$

Sendo que $E[I^2](x, y)$ é a média local do pixel de coordenadas (x, y) na imagem I^2 .

Por fim, o Limiar de Sauvola do pixel de coordenadas (x, y) na imagem I é:

$$T(x, y) = \mu(x, y) * [1 + k(\frac{\sigma(x, y)}{R} - 1)] \quad (14)$$

Sendo que R é o valor máximo teórico de σ e k é um parâmetro de sensibilidade, tipicamente 0.3.

2.5.3 Segmentação de caracteres

A segmentação de caracteres serve para dividir uma imagem contendo múltiplos caracteres em um grupo de imagens que contém apenas um caractere em cada. Partindo do pressuposto que a imagem original é binarizada, contendo portanto apenas pixels com valores de zero ou um, a segmentação de caracteres pode ser feita a partir da detecção de espaços brancos entre os caracteres (Casey; Lecolinet, 1996).

2.5.4 Filtragem de componentes conexos

A filtragem de componentes conexos é um processo que pode ser usado para isolar o objeto de interesse em uma imagem binarizada e remover ruído.

Ao binarizar uma imagem é comum que pixels que não pertencem ao objeto analisado acabem sendo classificados como parte do objeto, o que pode atrapalhar a análise da imagem. As vezes esses pixels estão conectados ao objeto, o que torna mais difícil identificá-los, mas as vezes estão desconectados, o que facilita sua identificação. Quando os pixels de ruído estão desconectados do objeto é possível identificá-los ao separar a imagem em *blobs*, regiões isoladas umas das outras.

Cada *blob* representa uma região da imagem na qual todos os pixels possuem valor 1 e possuem conectividade com pelo menos um outro pixel que também pertence àquele *blob*. A figura 10 exemplifica a separação de regiões da imagem em *blobs* conexos. Cada cor representa um *blob* diferente.



Figura 10 – Exemplo de separação de regiões por componentes conexos

A imagem separada em *blobs* pode ser representada como uma matriz na qual a cada posição é atribuído um número a depender do *blob* ao qual àquela posição pertence. Esses números são os rótulos das regiões.

Para realizar a filtragem, então basta determinar qual é o *blob* com maior área e eliminar os demais. A área de um *blob* nesse caso nada mais é do que a quantidade de pixels pertencentes à ele.

2.6 NEURAL PROCESSING UNIT

A unidade de processamento NPU (*Neural Processing Unit*) é um processador especializado em redes neurais. Ele é otimizado para operações matriciais e convolucionais, que são algumas das operações mais comuns nas redes neurais.

O intuito da NPU é permitir a execução de modelos de redes neurais de forma eficiente em aplicações embarcadas, que não costumam ser munidas de CPUs ou

GPUs poderosas. Para isso a NPU sacrifica flexibilidade em prol da performance em uma tarefa específica.

A arquitetura de uma NPU consiste basicamente em um grid de *processing engines* (PE), sendo que cada uma performa a computação de um neurônio (multiplicação, acumulação e sigmoide). A figura 11 mostra a arquitetura da NPU (Chen et al., 2020).

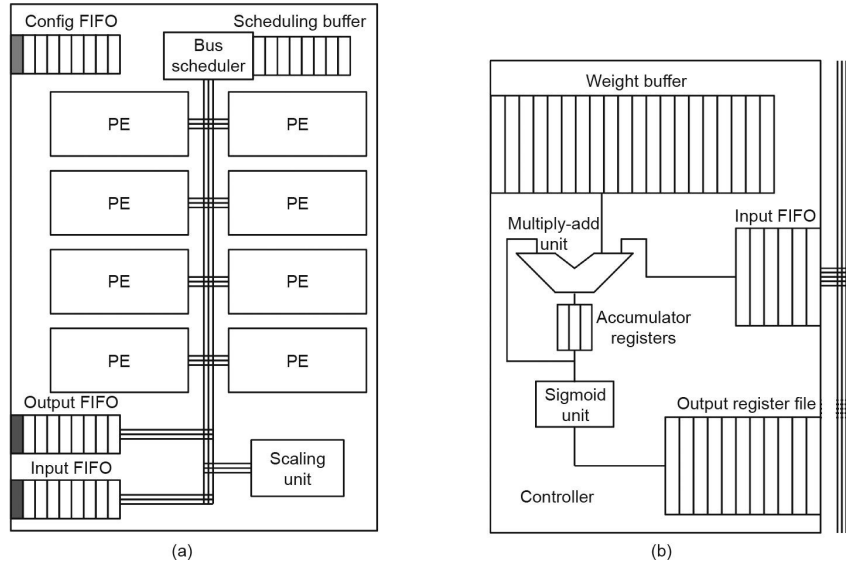


Figura 11 – A NPU. (a) NPU de 8 *processing engines*; (b) Fila de uma *processing engine* (Chen et al., 2020)

2.7 QUANTIZAÇÃO DE REDES NEURAIS

No contexto de redes neurais artificiais a quantização é o processo de converter os pesos e ativações da rede, que originalmente são valores reais (normalmente de 32 *bits*), para valores inteiros, normalmente de 8 *bits*. Esse processo reduz a precisão numérica, mas pode gerar ganhos significativos de performance, além de reduzir o espaço em memória ocupado pelo modelo (Jacob et al., 2018).

Em um possível esquema de quantização proposto por (Jacob et al., 2018) a relação entre um valor quantizado q e um valor real r é definida como:

$$r = S(q - Z) \quad (15)$$

Onde S e Z são os parâmetros de quantização, que são diferentes para cada vetor de pesos e ativações da rede. O parâmetro Z (*zero-point*) representa o valor de q quando $r = 0$, permitindo que o erro da quantização seja nulo para $r = 0$. Já a constante S (*scale*) é um valor real positivo.

É possível realizar a quantização após o treinamento do modelo, mas em muitos casos é recomendado que o modelo seja treinado já aplicando uma forma de quantização simulada, para que aprenda a compensar os erros de quantização.

Para se obter o valor quantizado q dado um valor real r é aplicado o seguinte algoritmo:

$$\text{clamp}(r; a, b) := \min(\max(r, a), b) \quad (16)$$

$$s(a, b, n) := \frac{b - a}{n - 1} \quad (17)$$

$$q(r; a, b, n) := \left\lfloor \frac{\text{clamp}(r; a, b) - a}{s(a, b, n)} \right\rfloor s(a, b, n) + a \quad (18)$$

Onde:

- r é o valor real sendo quantizado
- $[a; b]$ é o *range* de quantização, os valores mínimo e máximo esperados para r
- n é o número de níveis de quantização (256 para 8 *bits*)
- $\lfloor . \rfloor$ representa arredondamento para o inteiro mais próximo

Quantizando desta forma é possível obter um bom balanço entre performance e acurácia do modelo (Jacob et al., 2018).

3 DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS

Neste capítulo será discutido de forma mais aprofundada o problema que esse projeto se propõe a resolver. Isso envolve, além de uma observação detalhada do problema e seus requisitos, o contexto no qual o problema está inserido dentro da empresa e as motivações para resolvê-lo.

3.1 DESCRIÇÃO DA EMPRESA

A Palmsoft Tecnologia é uma empresa de desenvolvimento de software sob demanda. Produz sites, aplicativos, jogos, soluções de inteligência artificial e outros produtos de software.

Um dos produtos desenvolvidos pela Palmsoft é o *Aegis Access Control*. Uma plataforma que gerencia controle de acesso em condomínios equipados com fechaduras eletrônicas de desbloqueio por imagem facial.

O objetivo do *Aegis Access Control* é atender proprietários e gestores de apartamentos que fazem aluguel de curto prazo, permitindo que eles automatizem o processo de concessão de permissão de acesso aos imóveis. Quando o hóspede faz uma reserva, o gestor do apartamento acessa uma interface *web* onde ele cadastra o hóspede e as informações da reserva, como data de início e de fim. O hóspede então recebe um *email* com um *link* que o leva à uma página onde ele deve cadastrar uma foto do seu rosto, além de outras informações complementares, como placa do carro. Com base nas informações da reserva, o sistema então libera o acesso do hóspede ao condomínio e ao apartamento específico que foi alugado, apenas durante o período da estadia. Em cada porta do condomínio há um controlador de acesso facial, que destrava a fechadura da porta com base no rosto do usuário. O cadastro do rosto nos controladores de acesso é feito utilizando a foto enviada pelo usuário.

Para completar a experiência, o estacionamento é equipado com uma câmera com leitura automática de placas integrada, conectada ao portão. Da mesma forma que as imagens faciais são enviadas pelo sistema aos controladores de acesso das portas, as placas dos carros dos hóspedes também são enviadas às câmeras. Quando o carro do hóspede se aproxima, então, a câmera lê a placa do veículo, identifica que aquela placa possui acesso ao condomínio e envia um sinal ao portão para que ele abra.

Dessa forma, o hóspede recebe uma experiência completamente automatizada. Chega no condomínio e seu acesso já está liberado sem que precise ter contato com nenhum humano.

Além da experiência do hóspede, essa tecnologia é muito benéfica para os gestores dos apartamentos, que conseguem administrá-los de forma remota. Quando precisam conceder acesso a um prestador de serviço que vai limpar o apartamento,

por exemplo, isso também é feito com o mesmo processo.

3.2 MOTIVAÇÃO

Dentro do contexto do condomínio autônomo, as câmeras instaladas na garagem são um fator importante, que adiciona conforto à experiência do cliente. Essas câmeras são equipadas com tecnologia de visão computacional capaz de ler a placa de um veículo em questão de milissegundos, com ótima precisão.

Câmeras como essa já são parte do dia a dia de muitas pessoas, em diversas aplicações. Uma delas, muito comum, é a de controle de acesso em estacionamentos pagos, como em shoppings ou supermercados. Elas permitem que o sistema registre a placa do carro de cada cliente e a associe ao seu *ticket* de estacionamento. Uma das vantagens disso é que caso o cliente perca o *ticket* impresso o sistema poderá gerar um novo sem problemas, mantendo o mesmo horário de entrada. Outra vantagem é que quando o cliente se aproxima da cancela de saída a placa do seu carro é rapidamente detectada e o sistema pode conferir se aquele cliente pagou o estacionamento sem que ele precise utilizar o *ticket* impresso na saída. Isso evita congestionamentos na saída e melhora o fluxo de veículos ao poupar tempo. Esse sistema também pode ser utilizado para conferir acesso gratuito à funcionários, permitindo a entrada deles sem retirada de *ticket* com base na placa do carro.

Outra aplicação para esse tipo de câmera é em sistemas de segurança em estacionamentos. Ao detectar placas de carros o sistema pode gerar *logs* de entrada e saída de veículos. Isso ajuda muito na auditoria dos vídeos das câmeras de segurança do estacionamento, pois os momentos mais relevantes ficam registrados. Dessa maneira fica fácil responder perguntas como: "Em qual data e hora o carro com a placa X acessou o estacionamento?". Além disso é possível identificar facilmente acesso de veículos não autorizados a um estacionamento privado.

No condomínio que utiliza a solução do *Aegis Access Control* a câmera utilizada é um dispositivo da Intelbras, modelo VIP 5460 LPR IA, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500. O alto valor desse dispositivo motivou a exploração dessa tecnologia para desenvolvimento de alternativas mais acessíveis, que possam ser utilizadas em aplicações onde o custo é um fator determinante.

Um outro fator motivador é que a Palmsoft Tecnologia demonstrou interesse em testar a performance de *hardware* de baixo custo para aplicações de inteligência artificial, com o objetivo de validar o equipamento para desenvolvimento de produtos nessa área. Para isso, foi adquirida uma placa Rock 3A, da radxa, equipada com uma NPU de 0.8 TOPs.

Portanto, juntando a motivação do desenvolvimento de uma alternativa barata para câmera com leitura automática de placas de carro com a de validar uma placa de baixo custo para aplicações de inteligência artificial, foi pensado um projeto que

desenvolvesse um modelo de leitura de placas de carro e validasse sua execução na placa Rock 3A.

O objetivo desse projeto, portanto, não é criar um protótipo funcional, mas criar e validar um algoritmo de leitura de placas de carro, que recebe uma imagem e retorna os caracteres da placa do veículo contido na imagem (se houver), e analisar a performance desse algoritmo ao ser executado na placa Rock 3A, validando assim a possibilidade da criação de um produto que utilize o algoritmo criado nesse *hardware*.

3.3 ANÁLISE DO PROBLEMA

O objetivo geral deste projeto pode ser definido como:

"Desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de uma solução de baixo custo que execute esse software em uma placa Rock 3A."

O problema, portanto pode ser dividido em duas partes:

- Desenvolver o *software* de leitura automática de placas de veículos
- Validar a placa Rock 3A como um *hardware* viável para execução desse *software*

A primeira parte envolve o desenvolvimento de um algoritmo complexo de visão computacional capaz de ler os caracteres da placa de um carro ao longo de múltiplas etapas, exigindo conhecimento técnico acerca dos diferentes modelos de inteligência artificial para processamento de imagens e técnicas de visão computacional.

Já a segunda parte envolve executar e avaliar a performance do algoritmo no *hardware*. Para isso é necessária compreensão do dispositivo a ser utilizado, os requisitos para executar modelos de redes neurais na NPU e as configurações necessárias.

3.3.1 Criação do algoritmo de leitura automática de placas de carros

O *software* de leitura automática de placas de veículos a ser desenvolvido nada mais é do que um algoritmo que recebe uma imagem como entrada e devolve uma sequência de caracteres correspondente à placa do carro contido na imagem (se houver). Esse algoritmo deve ser possível de ser executado na placa Rock 3A, portanto deve se observar as limitações desse *hardware* que serão discutidas posteriormente.

As características desejáveis do algoritmo são principalmente:

- **Acurácia:** O algoritmo deve inferir corretamente a maioria das placas dos carros contidos nas imagens entregues a ele. Um leitor automático de placas que comete muitos erros não tem utilidade, portanto a proporção de placas inferidas

corretamente em relação ao total de imagens analisadas deve ser o mais alta possível.

- **Velocidade:** Para muitas aplicações a velocidade é crítica, especialmente quando se deseja inferir a placa de carros passantes. Portanto, o tempo necessário para que o algoritmo processe uma imagem e retorne o resultado deve ser mínimo.

3.3.2 Teste do algoritmo desenvolvido na placa Rock 3A

3.4 REQUISITOS DA SOLUÇÃO

4 SOLUÇÃO PROPOSTA E METODOLOGIA UTILIZADA

4.1 ANÁLISE DOS DADOS

4.1.1 Características do objeto

4.1.2 Características do *dataset* utilizado

4.2 PIPELINE DE LEITURA DE PLACAS

4.3 MODELO DE DETECÇÃO DE OBJETOS

4.4 ALGORITMO DE EXTRAÇÃO DE CARACTERES

4.5 MODELOS DE CLASSIFICAÇÃO DE CARACTERES

4.6 METODOLOGIA DE CONSTRUÇÃO

5 DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS

5.1 PREPARAÇÃO DOS DADOS

5.2 TREINAMENTO DO MODELO DE DETECÇÃO DE PLACAS

5.3 IMPLEMENTAÇÃO DO ALGORITMO DE EXTRAÇÃO DE CARACTERES

5.4 CRIAÇÃO DO DATASET DE CARACTERES

5.5 TREINAMENTO DOS MODELOS DE CLASSIFICAÇÃO DE CARACTERES

5.6 QUANTIZAÇÃO E TRANSFORMAÇÃO DOS MODELOS PARA FORMATO COMPATÍVEL COM O *HARDWARE*

5.7 PREPARAÇÃO DO *HARDWARE*

5.8 TESTES REALIZADOS

6 CONCLUSÃO

Instruções da Coordenação do PFC:

A Conclusão deve apresentar:

- Uma **síntese** do problema tratado, da solução proposta e dos principais resultados obtidos;
- Uma discussão sobre o que foi atingido no PFC em relação aos objetivos inicialmente traçados e sobre as limitações encontradas;
- Uma discussão sobre possíveis aprimoramentos;
- Destacar os impactos do PFC para a empresa/clientes da empresa/instituto de pesquisa, além de possíveis impactos organizacionais, tecnológicos, financeiros, éticos, ecológicos, etc.
- Sugestão de trabalhos futuros (como se poderia dar continuidade ao PFC; aplicar o desenvolvimento realizado no PFC a outros problemas/processos; etc)

REFERÊNCIAS

CASEY, Richard G.; LECOLINET, Eric. A Survey of Methods and Strategies in Character Segmentation. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 18, n. 7, p. 690–706, 1996. Revisão abrangente de técnicas de segmentação, incluindo histograma de projeção vertical.

CHAPMAN, Peter; CLINTON, Julian; KERBER, Randy; KHABAZA, Thomas; REINARTZ, Thomas; SHEARER, Colin; WIRTH, Rüdiger. **CRISP-DM 1.0: Step-by-step data mining guide**. [S.l.], 2000. Technical report. Disponível em: <https://www.crisp-dm.org>.

CHEN, Yunji; XIE, Yuan; SONG, Linghao; CHEN, Fan; TANG, Tianshi. A Survey of Accelerator Architectures for Deep Neural Networks. **Engineering**, Elsevier, v. 6, n. 3, p. 264–274, 2020. Panorama de aceleradores de DNN incluindo NPUs embarcadas como as encontradas em SoCs móveis e de borda.

FAYYAD, Usama M.; PIATETSKY-SHAPIRO, Gregory; SMYTH, Padhraic. From Data Mining to Knowledge Discovery in Databases. **AI Magazine**, v. 17, n. 3, p. 37–54, 1996. DOI: 10.1609/aimag.v17i3.1230. Disponível em: <https://ojs.aaai.org/index.php/aimagazine/article/view/1230>.

IBM INC. **What are convolutional neural networks?** [S.l.: s.n.], 2024. <https://www.ibm.com/>. Acessado em março de 2026. Disponível em: <https://www.ibm.com/think/topics/convolutional-neural-networks>.

JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, p. 2704–2713, 2018. Artigo seminal sobre quantização int8 com zero-point; base do esquema usado no TFLite e no RKNN toolkit.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet Classification with Deep Convolutional Neural Networks. In: **ADVANCES in Neural Information Processing Systems 25 (NIPS 2012)**. [S.l.: s.n.], 2012. p. 1097–1105. Disponível em: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>.

LECUN, Yann; BOTTOU, Léon; BENGIO, Yoshua; HAFFNER, Patrick. Gradient-Based Learning Applied to Document Recognition. **Proceedings of the IEEE**, v. 86, n. 11, p. 2278–2324, 1998. DOI: 10.1109/5.726791. Disponível em: <https://doi.org/10.1109/5.726791>.

REDMON, Joseph; DIVVALA, Santosh; GIRSHICK, Ross; FARHADI, Ali. You Only Look Once: Unified, Real-Time Object Detection. In: **PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2016. p. 779–788.

RUMELHART, David E.; HINTON, Geoffrey E.; WILLIAMS, Ronald J. Learning Representations by Back-propagating Errors. **Nature**, Nature Publishing Group, v. 323, p. 533–536, 1986. Artigo seminal que formalizou o algoritmo de backpropagation.

SAHA, Sumit. A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way. **Medium**, 2018.

SAUVOLA, Jaakko; PIETIKÄINEN, Matti. Adaptive Document Image Binarization. **Pattern Recognition**, Elsevier, v. 33, n. 2, p. 225–236, 2000. Referência para binarização adaptativa local, análoga à abordagem por faixas do projeto.

THEAILEARNER. **Perspective Transformation**. [S.l.: s.n.], 2023.
<https://theailearner.com>. Acessado em março de 2026. Disponível em:
<https://theailearner.com/tag/cv2-getperspectivetransform/>.

ZOU, Zhengxia; SHI, Zhenwei; GUO, Yuhong; YE, Jieping. Object Detection in 20 Years: A Survey. **arXiv preprint arXiv:1905.05055**, 2019.

APÊNDICE A – DESCRIÇÃO 1

Textos elaborados pelo autor, a fim de completar a sua argumentação. Deve ser precedido da palavra APÊNDICE, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.

ANEXO A – DESCRIÇÃO 2

São documentos não elaborados pelo autor que servem como fundamentação (mapas, leis, estatutos). Deve ser precedido da palavra ANEXO, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.